Multivariate_Model_Forecast

2026-04-16

Multivariate Model

We now switch models, and we'll use a multivariate model. Ideally we are going to use a VAR model, in which there will be 2 series : one will be the BTP-Bund Spread, and the other will be one between the EURO STR series or the EuroStoxx Banks series. If we have time, we could also use a trivariate VAR model, incorporating them all.

```
data <- read_excel("Spread_daily_17_04.xlsx")
data <- data |> na.omit() |> rename(date = Date, spread = `IT10DE10=TWEB (BID)`)
data$spread <- data$spread * -1
spread  <- xts(data[, -1], order.by = as.Date(data$date, format = "%d/%m/%y"))
```

Inspect data

```
head(spread)
```

```
##            spread
## 2021-04-19  102.4
## 2021-04-20  104.1
## 2021-04-21  102.1
## 2021-04-22  102.0
## 2021-04-23  104.7
## 2021-04-26  106.0
```

```
str(spread)
```

```
## An xts object on 2021-04-19 / 2026-04-17 containing:
##   Data:    double [1275, 1]
##   Columns: spread
##   Index:   Date [1275] (TZ: "UTC")
```

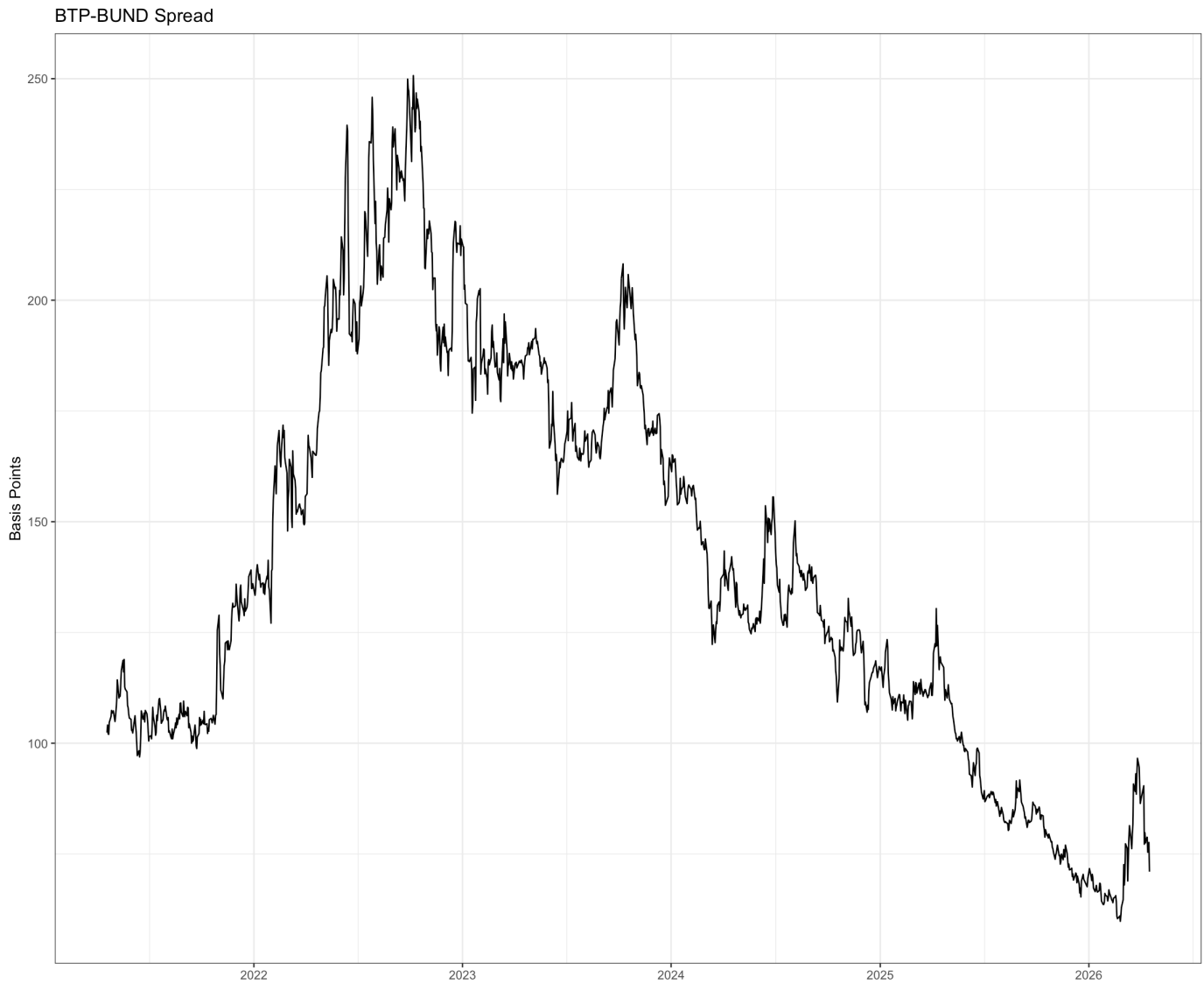
```
sum(is.na(spread))
```

```
## [1] 0
```

```
any(is.infinite(spread))
```

```
## [1] FALSE
```

Plot the data




```
##                   spread
## nobs          1275.000000
## NAs              0.000000
## Minimum         59.800000
## Maximum        250.700000
## 1. Quartile    105.700000
## 3. Quartile    178.300000
## Mean           140.677255
## Median         133.900000
## Sum         179363.500000
## SE Mean          1.272432
## LCL Mean       138.180962
## UCL Mean       143.173548
## Variance      2064.332386
## Stdev           45.434925
## Skewness         0.272696
## Kurtosis        -0.854207
```

```
risultato_adf <- adfTest(spread, lags = 1, type = "c")
print(risultato_adf)
```

```
##
## Title:
##  Augmented Dickey-Fuller Test
##
## Test Results:
##   PARAMETER:
##     Lag Order: 1
##   STATISTIC:
##     Dickey-Fuller: -1.1559
##   P VALUE:
##     0.6287
##
## Description:
##  Fri Apr 17 20:56:50 2026 by user:
```

```
# Equivalent with auto determination of lags

adf_test <- function(serie, nome_serie = "Serie") {
  cat("\n==============================================\n")
  cat("Stationarity test (ADF):", nome_serie, "\n")
  cat("==============================================\n")
  types <- c("none", "drift", "trend")
  results <- lapply(types, function(t) {
    test <- ur.df(serie, type = t, selectlags = "AIC")
    stat <- test@teststat[1]
    val <- test@cval[1, "5pct"]
    result <- if (stat < val) "Stationary" else "NOT Stationary"
    return(data.frame(
      Type = t,
      ADF_stat = round(stat, 3),
      Tau_5pct = round(val, 3),
      Result = result,
      stringsAsFactors = FALSE
    ))
  })
  table <- bind_rows(results)
  print(table)
}

adf_test(spread, "Spread BTP/Bund")
```

```
##
## ================================================
## Stationarity test (ADF): Spread BTP/Bund
## ================================================
##    Type ADF_stat Tau_5pct         Result
## 1  none   -0.594    -1.95 NOT Stationary
## 2 drift   -1.156    -2.86 NOT Stationary
## 3 trend   -2.152    -3.41 NOT Stationary
```

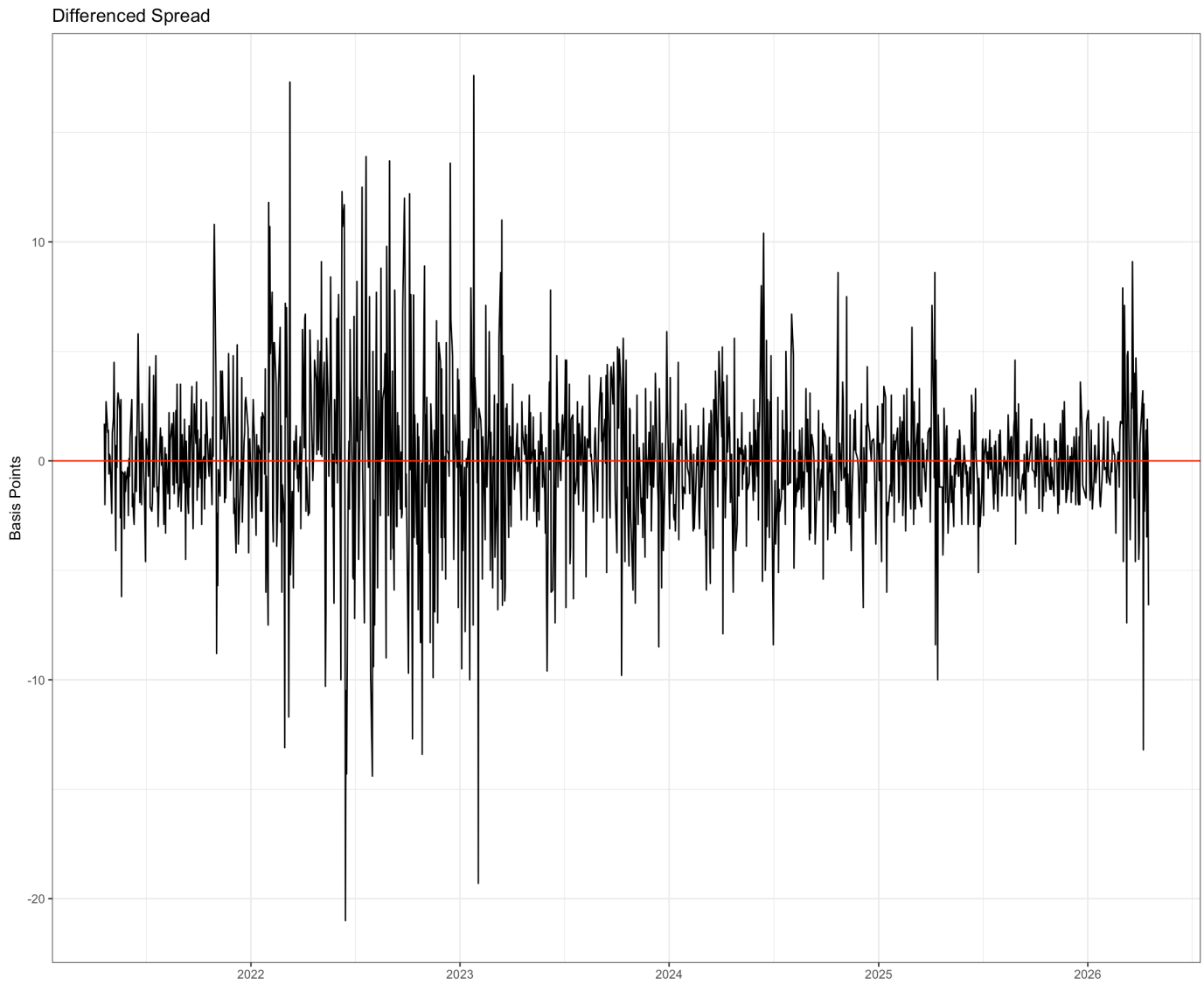
The series is not stationary, so we need to differenciate it to use it in a VAR model.

```
d_spread <- diff(spread) |> na.omit() # we loose the first observation due to differe
ncing
#d_train <- d_spread[1:n_train]
#d_test  <- d_spread[(n_train + 1):nrow(d_spread)]

#cat("\nTraining observations:", nrow(d_train))
#cat("\nTest observations:", nrow(d_test))
#cat("\nTrain period:", as.character(index(d_train)[1]), "->", as.character(index(d_t
rain)[nrow(d_train)]))
#cat("\nTest period:", as.character(index(d_test)[1]), "->", as.character(index(d_tes
t)[nrow(d_test)]),"\n")
```

```
adf_test(d_spread, "Differenced spread BTP/Bund")
```

```
##
## ================================================
## Stationarity test (ADF): Differenced spread BTP/Bund
## ================================================
##    Type ADF_stat Tau_5pct     Result
## 1  none  -25.433    -1.95 Stationary
## 2 drift  -25.424    -2.86 Stationary
## 3 trend  -25.477    -3.41 Stationary
```

now the train series is stationary



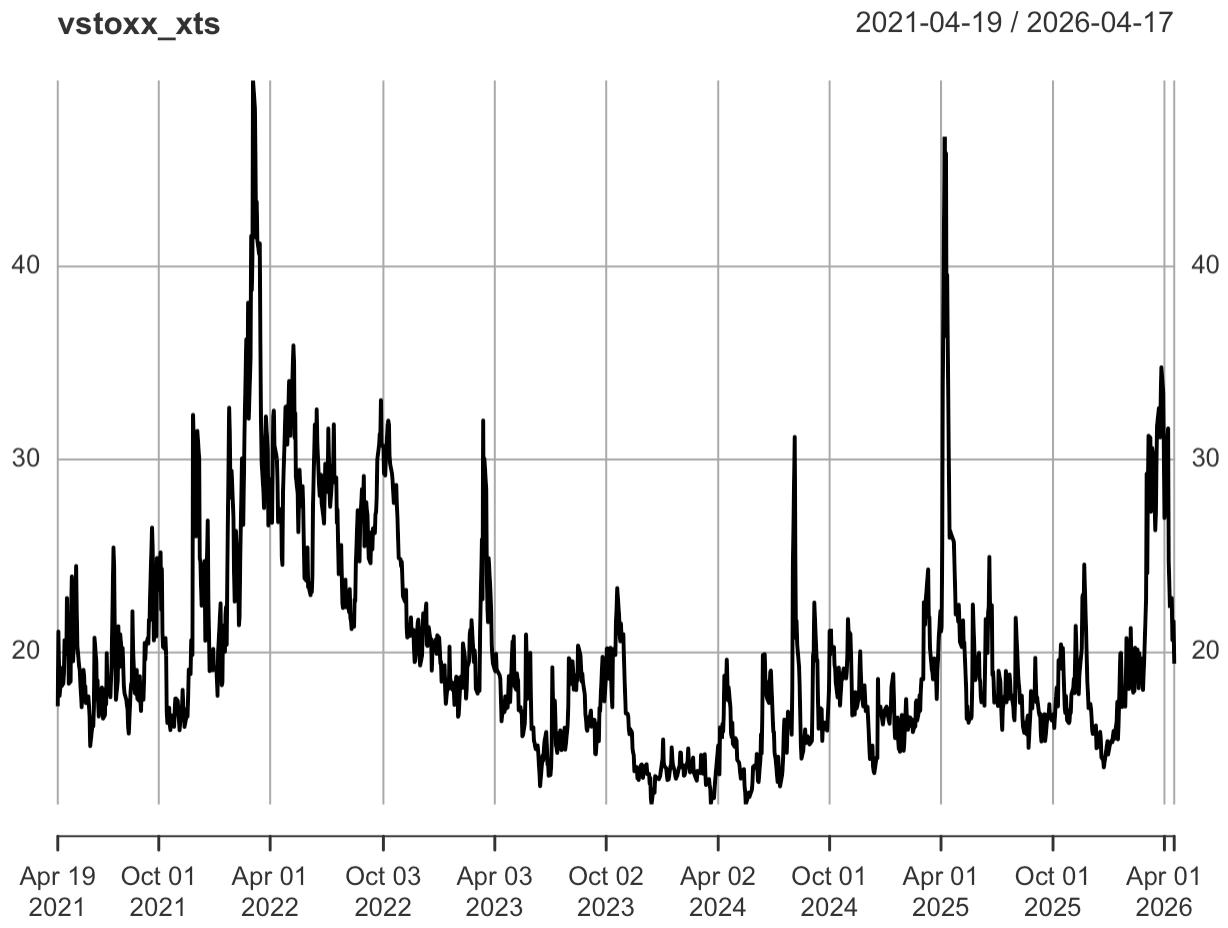
Multicollinearity Test

We now test for multicollinearity, so now we load the 2 other series.

```
# ==========================================
# PHASE 1: EXCEL DATA ACQUISITION & ALIGNMENT
# ==========================================

# Step 1: Load required libraries
#install.packages("readxl")
getwd()
```

```
## [1] "/Users/jacopo/Downloads"
```

```
library(readxl)
library(xts)
library(zoo)

# Importa l'indice VStoxx Europe Volatility

vstoxx_raw <- read_excel("vstoxx_17_04.xlsx")
vstoxx_raw <- vstoxx_raw[-1,]

# 3. Trasforma in Serie Storica
# Assumendo che la Colonna 1 sia la Data e la Colonna 3 sia il Tasso
#estr_xts <- xts(as.numeric(estr_raw[[3]]), order.by = as.Date(estr_raw[[1]]))
#colnames(estr_xts) <- "ESTR"
#estr_clean <- na.remove(estr_xts)

vstoxx_xts <- xts(as.numeric(vstoxx_raw[[2]]), order.by = as.Date(vstoxx_raw[[1]]))
colnames(vstoxx_xts) <- "VStoxx"
vstoxx_xts <- na.omit(vstoxx_xts)

plot(vstoxx_xts, type="l")
```




```
# ADF Test for vstoxx data
adf.test(vstoxx_xts)
```

```
## Warning in adf.test(vstoxx_xts): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  vstoxx_xts
## Dickey-Fuller = -4.2768, Lag order = 10, p-value = 0.01
## alternative hypothesis: stationary
```

```
# p-value = 0.01 -> Already stationary

# Merge the datasets
merged_data_2 <- merge(d_spread, vstoxx_xts, join = "inner")
head(merged_data_2)
```

```
##            spread  VStoxx
## 2021-04-20    1.7 21.0797
## 2021-04-21   -2.0 19.2813
## 2021-04-22   -0.1 17.7562
## 2021-04-23    2.7 18.1883
## 2021-04-26    1.3 18.2841
## 2021-04-27    1.4 19.2123
```

```
clean_data_2 <- na.omit(merged_data_2)

# ========================
# 4. Split train/test 80/20
# ========================
ntest  <- round(nrow(clean_data_2) * 0.2)
ntrain <- nrow(clean_data_2) - ntest

train <- clean_data_2[1:ntrain, ]
test  <- clean_data_2[(ntrain + 1):nrow(clean_data_2), ]

cat("Training observations:", nrow(train), "\n")
```

```
## Training observations: 1018
```

```
cat("Test observations:", nrow(test), "\n")
```

```
## Test observations: 255
```

```
cat("Train period:", as.character(index(train)[1]), "-", as.character(tail(index(trai
n), 1)), "\n")
```

```
## Train period: 2021-04-20 - 2025-04-10
```

```
cat("Test period:", as.character(index(test)[1]), "-", as.character(tail(index(test),
1)), "\n")
```

```
## Test period: 2025-04-11 - 2026-04-17
```

Now we should have a matrix with the d_spread value and the vstoxx index value, with matching data. We also have the train and test matrices.

MQ Test

We have both series stationary, we now run the qm test to see if we need a VAR model.

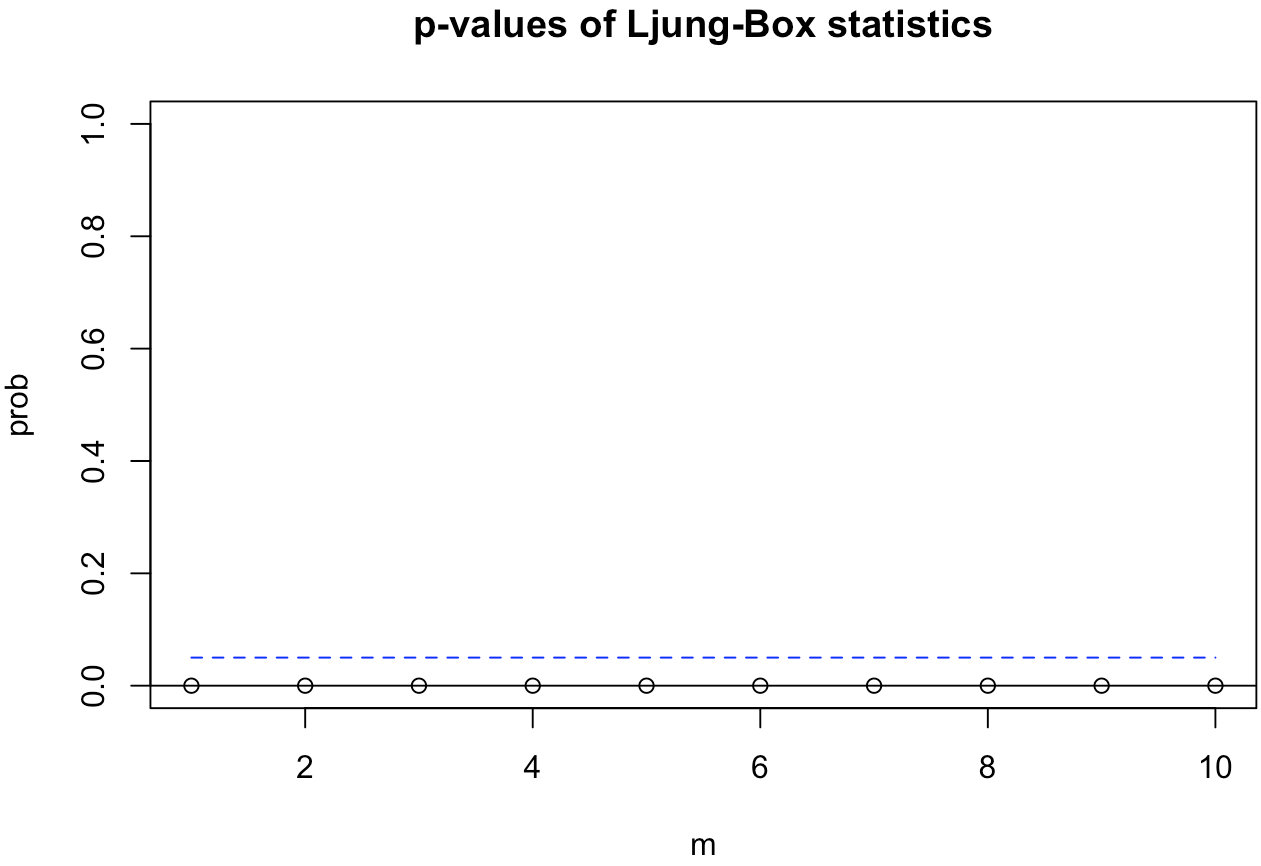
```
library(MTS)
library(vars)
```

```
##
## Attaching package: 'vars'
```

```
## The following object is masked from 'package:MTS':
##
##     VAR
```

```
# MQ test
mq(train, lag=10)
```

```
## Ljung-Box Statistics:
##         m       Q(m)     df    p-value
##  [1,]     1       932       4        0
##  [2,]     2      1782       8        0
##  [3,]     3      2572      12        0
##  [4,]     4      3295      16        0
##  [5,]     5      3968      20        0
##  [6,]     6      4596      24        0
##  [7,]     7      5188      28        0
##  [8,]     8      5748      32        0
##  [9,]     9      6277      36        0
## [10,]    10      6782      40        0
```




```
# We have all lags in which p-value is less than 0.05 ( equal to 0), so we can estima
te a VAR model.
```

VAR Model Estimation

```
# 1. Determine optimal VAR lag length
lag_selection <- VARselect(train, lag.max = 10, type = "const")
lag_selection
```

```
## $selection
## AIC(n)  HQ(n)  SC(n) FPE(n)
##      2      1      1      2
##
## $criteria
##                1         2         3         4         5         6         7
## AIC(n)  3.713404  3.707111  3.707168  3.708912  3.714457  3.714927  3.717523
## HQ(n)   3.724521  3.725639  3.733106  3.742262  3.755218  3.763099  3.773106
## SC(n)   3.742665  3.755878  3.775441  3.796693  3.821745  3.841721  3.863824
## FPE(n) 40.993130 40.735961 40.738271 40.809418 41.036389 41.055688 41.162496
##                8         9        10
## AIC(n)  3.720049  3.724829  3.722992
## HQ(n)   3.783043  3.795234  3.800808
## SC(n)   3.885857  3.910144  3.927814
## FPE(n) 41.266681 41.464515 41.388542
```

```
# We have optimal lag = 1 chosen by all the models
library(vars)
# Fit the model
var_fit <- VAR(train, p = 2, type = "const")
summary(var_fit)
```

```
##
## VAR Estimation Results:
## =========================
## Endogenous variables: spread, VStoxx
## Deterministic variables: const
## Sample size: 1016
## Log Likelihood: -4750.214
## Roots of the characteristic polynomial:
## 0.9677 0.1795 0.1795 0.1395
## Call:
## VAR(y = train, p = 2, type = "const")
##
##
## Estimation results for equation spread:
## =======================================
## spread = spread.l1 + VStoxx.l1 + spread.l2 + VStoxx.l2 + const
##
##           Estimate Std. Error t value Pr(>|t|)
## spread.l1  0.05103    0.03287   1.553   0.1208
## VStoxx.l1 -0.13457    0.07600  -1.771   0.0769 .
## spread.l2 -0.02387    0.03165  -0.754   0.4510
## VStoxx.l2  0.13289    0.07643   1.739   0.0824 .
## const      0.05511    0.45102   0.122   0.9028
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 3.899 on 1011 degrees of freedom
## Multiple R-Squared: 0.004865,    Adjusted R-squared: 0.0009276
## F-statistic: 1.236 on 4 and 1011 DF,  p-value: 0.294
##
##
## Estimation results for equation VStoxx:
## =======================================
## VStoxx = spread.l1 + VStoxx.l1 + spread.l2 + VStoxx.l2 + const
##
##           Estimate Std. Error t value             Pr(>|t|)
## spread.l1  0.03199    0.01427   2.241             0.025227 *
## VStoxx.l1  0.84872    0.03300  25.719 < 0.0000000000000002 ***
## spread.l2  0.01208    0.01374   0.879             0.379492
## VStoxx.l2  0.11500    0.03319   3.465             0.000552 ***
## const      0.74982    0.19585   3.829             0.000137 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 1.693 on 1011 degrees of freedom
## Multiple R-Squared: 0.9154,  Adjusted R-squared: 0.9151
## F-statistic:  2736 on 4 and 1011 DF,  p-value: < 0.00000000000000022
##
##
```

```
##
## Covariance matrix of residuals:
##        spread VStoxx
## spread 15.201  1.931
## VStoxx  1.931  2.866
##
## Correlation matrix of residuals:
##        spread VStoxx
## spread 1.0000 0.2925
## VStoxx 0.2925 1.0000
```

VAR Diagnostics

```
# ==========================================
# PHASE 4: VAR DIAGNOSTICS
# ==========================================

# 1. Test for leftover Serial Correlation (Portmanteau Test)
# We test up to 10 lags to be safe. We want p-value > 0.05
serial_test <- serial.test(var_fit, lags.pt = 10, type = "PT.asymptotic")
cat("\n--- Serial Correlation Test ---\n")
```

```
##
## --- Serial Correlation Test ---
```

```
print(serial_test)
```

```
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object var_fit
## Chi-squared = 44.6, df = 32, p-value = 0.06854
```

```
## $serial
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object var_fit
## Chi-squared = 44.6, df = 32, p-value = 0.06854
```

```
# 2. Test for ARCH Effects (Volatility Clustering)
# We want p-value < 0.05 to justify using DCC-GARCH next
arch_test <- arch.test(var_fit, lags.multi = 5, multivariate.only = TRUE)
cat("\n--- ARCH Effects Test ---\n")
```

```
##
## --- ARCH Effects Test ---
```

```
print(arch_test)
```

```
##
##  ARCH (multivariate)
##
## data:  Residuals of VAR object var_fit
## Chi-squared = 386.74, df = 45, p-value < 0.00000000000000022
```

```
## $arch.mul
##
##  ARCH (multivariate)
##
## data:  Residuals of VAR object var_fit
## Chi-squared = 386.74, df = 45, p-value < 0.00000000000000022
```

```
## We need the DCC-GARCH model, as there are arch effects (p-value is near 0)
```

DCC-GARCH Model

```
# ==========================================
# PHASE 5: THE DCC-GARCH MODEL
# ==========================================
library(rugarch)
library(rmgarch)
```

```
##
## Attaching package: 'rmgarch'
```

```
## The following objects are masked from 'package:dplyr':
##
##     first, last
```

```
## The following objects are masked from 'package:xts':
##
##     first, last
```

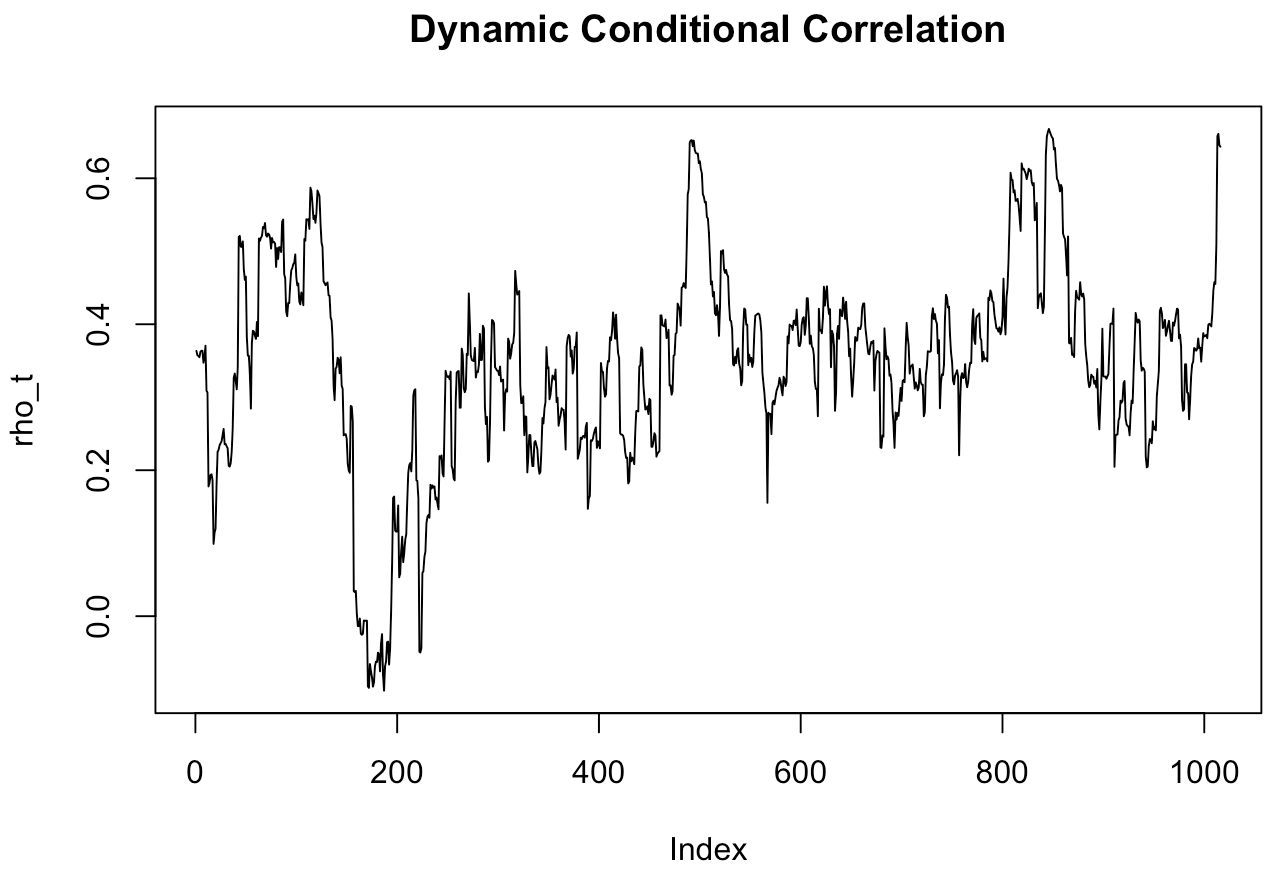
```
# 1. Extract the residuals from your fitted VAR model
# The scale (* 100) is crucial! GARCH optimizers fail if the numbers are too tiny
(e.g. 0.0001)
var_residuals <- residuals(var_fit) * 100

# Convert the residuals back into a Time Series (xts) object for plotting later
# We use the dates from diff_data, but remember the VAR loses the first 'p' rows
# So we align the dates by tailing the diff_data
var_residuals_xts <- xts(var_residuals, order.by = tail(index(train), nrow(var_residu
als)))
colnames(var_residuals_xts) <- c("Spread_Shock", "VStoxx_Shock")

# 2. Specify the Univariate GARCH Models
# Because the VAR already handled the mean, we set the ARMA order to (0,0) with NO me
an!
uspec <- ugarchspec(
  mean.model = list(armaOrder = c(0, 0), include.mean = FALSE),
  variance.model = list(garchOrder = c(1, 1), model = "sGARCH"),
  distribution.model = "norm" # You can change to "std" (Student-t) if it fails to co
nverge
)

# Replicate this specification for both of our variables
mspec <- multispec(replicate(2, uspec))

# 3. Specify the Multivariate DCC-GARCH model
# We set VAR = FALSE because we ALREADY did the VAR manually and are feeding it the r
esiduals!
dcc_spec <- dccspec(
  uspec = mspec,
  VAR = FALSE,
  dccOrder = c(1, 1),
  distribution = "mvnorm"
)

# 4. Fit the DCC-GARCH model!
# We feed it our scaled VAR residuals
cat("\nFitting DCC-GARCH...\n")
```

```
##
## Fitting DCC-GARCH...
```

```
fit_dcc <- dccfit(dcc_spec, data = var_residuals_xts)

# Print the final estimated model
show(fit_dcc)
```

```
##
## *---------------------------------*
## *          DCC GARCH Fit          *
## *---------------------------------*
##
## Distribution         :  mvnorm
## Model                :  DCC(1,1)
## No. Parameters       :  9
## [VAR GARCH DCC UncQ] : [0+6+2+1]
## No. Series           :  2
## No. Obs.             :  1016
## Log-Likelihood       :  -13744.82
## Av.Log-Likelihood    :  -13.53
##
## Optimal Parameters
## ------------------------------------
##                          Estimate  Std. Error  t value Pr(>|t|)
## [Spread_Shock].omega  5034.745386 2124.437986   2.3699 0.017792
## [Spread_Shock].alpha1    0.186042    0.050016   3.7197 0.000199
## [Spread_Shock].beta1     0.793319    0.050339  15.7595 0.000000
## [VStoxx_Shock].omega  2705.186017  970.850256   2.7864 0.005330
## [VStoxx_Shock].alpha1    0.357349    0.089051   4.0128 0.000060
## [VStoxx_Shock].beta1     0.597306    0.062259   9.5939 0.000000
## [Joint]dcca1             0.041319    0.013726   3.0102 0.002611
## [Joint]dccb1             0.944120    0.016263  58.0526 0.000000
##
## Information Criteria
## ---------------------
##
## Akaike       27.074
## Bayes        27.118
## Shibata      27.074
## Hannan-Quinn 27.091
##
##
## Elapsed time : 0.4461708
```

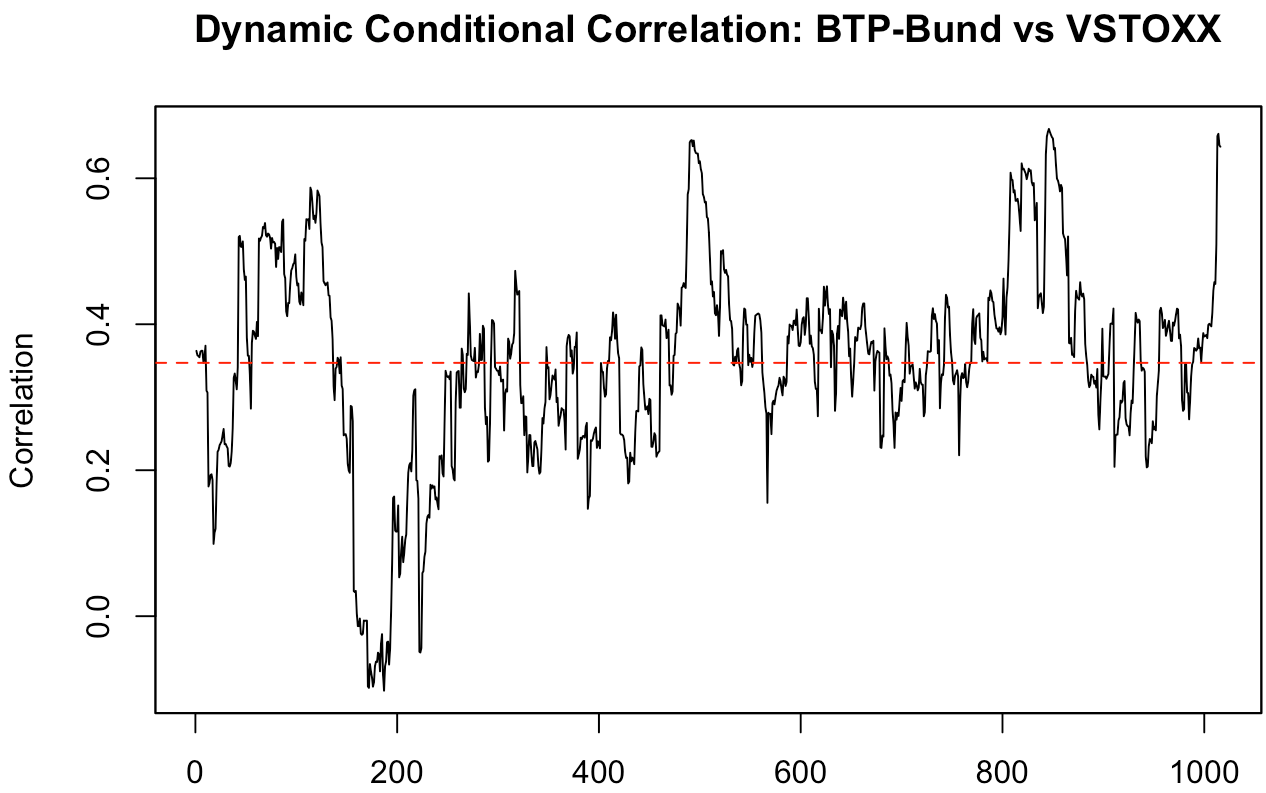
```
# Dynamic conditional correlations over time
dcc_corr <- rcor(fit_dcc)        # array: 2x2xT
rho_t    <- dcc_corr[1,2,]       # time series of rho(BTP-Bund, VSTOXX)
plot(rho_t, type = "l", main = "Dynamic Conditional Correlation")
```




```
# Conditional volatilities
vol <- sigma(fit_dcc)

# DCC parameters (alpha + beta, should sum < 1)
coef(fit_dcc)
```

```
##  [Spread_Shock].omega [Spread_Shock].alpha1  [Spread_Shock].beta1
##         5034.74538580            0.18604178            0.79331885
##  [VStoxx_Shock].omega [VStoxx_Shock].alpha1  [VStoxx_Shock].beta1
##         2705.18601731            0.35734887            0.59730636
##          [Joint]dcca1          [Joint]dccb1
##            0.04131896            0.94412007
```

```
library(xts)

rho_t2 <- rcor(fit_dcc)[1, 2, ]
resids <- residuals(var_fit)

plot(zoo(rho_t2, order.by = index(resids)), type = "l",
     main = "Dynamic Conditional Correlation: BTP-Bund vs VSTOXX",
     ylab = "Correlation", xlab = "")
abline(h = mean(rho_t2), col = "red", lty = 2)
```

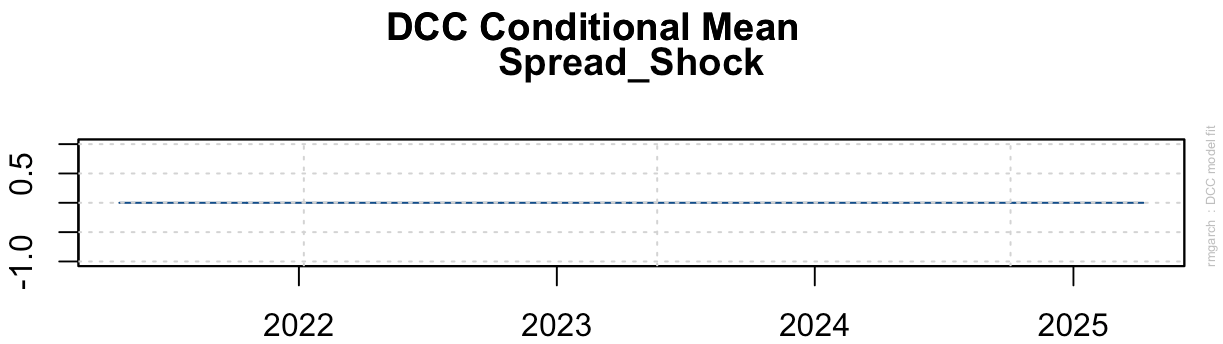


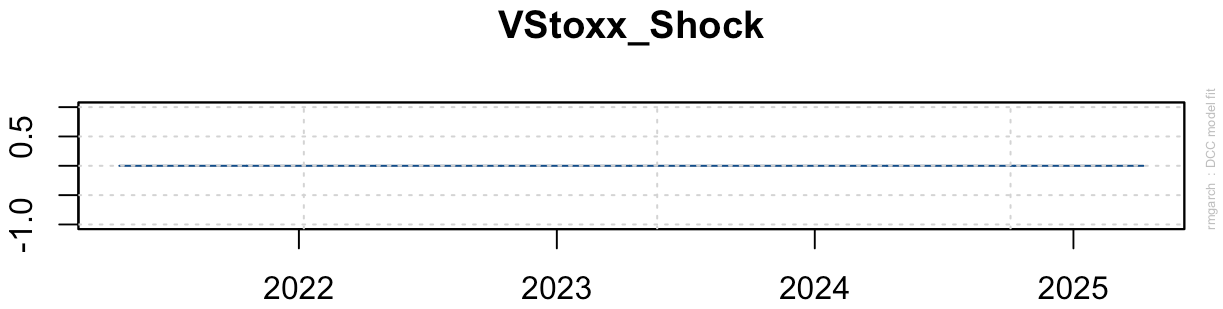
Model Diagnostics

```
# 1. Model Diagnostics
# Standardized residuals should have no ARCH effects remaining
infocriteria(fit_dcc)   # AIC, BIC
```

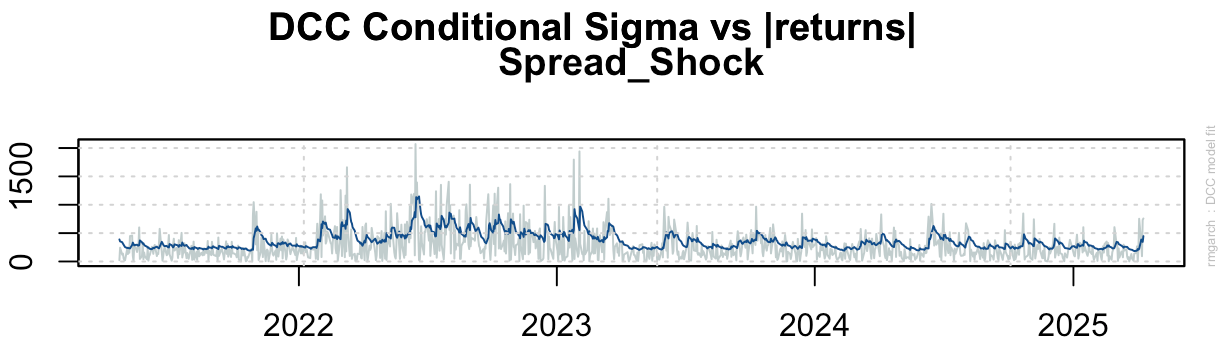
```
##
## Akaike       27.07445
## Bayes        27.11807
## Shibata      27.07430
## Hannan-Quinn 27.09102
```

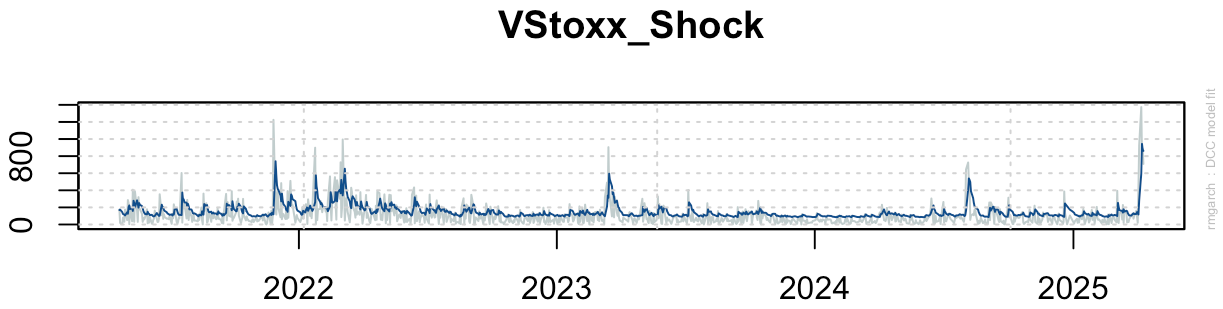
```
plot(fit_dcc, which=1)          # rmgarch gives several diagnostic plots
```



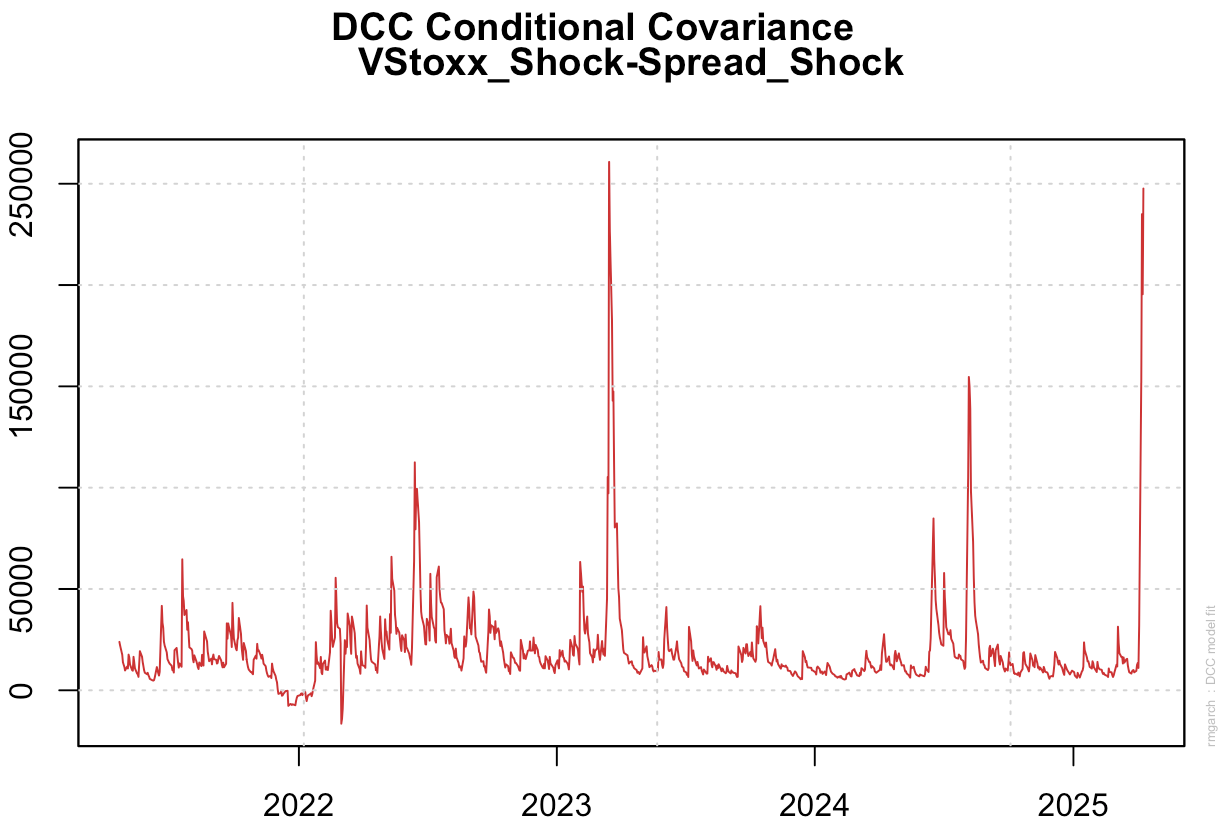



```
plot(fit_dcc, which=2)
```

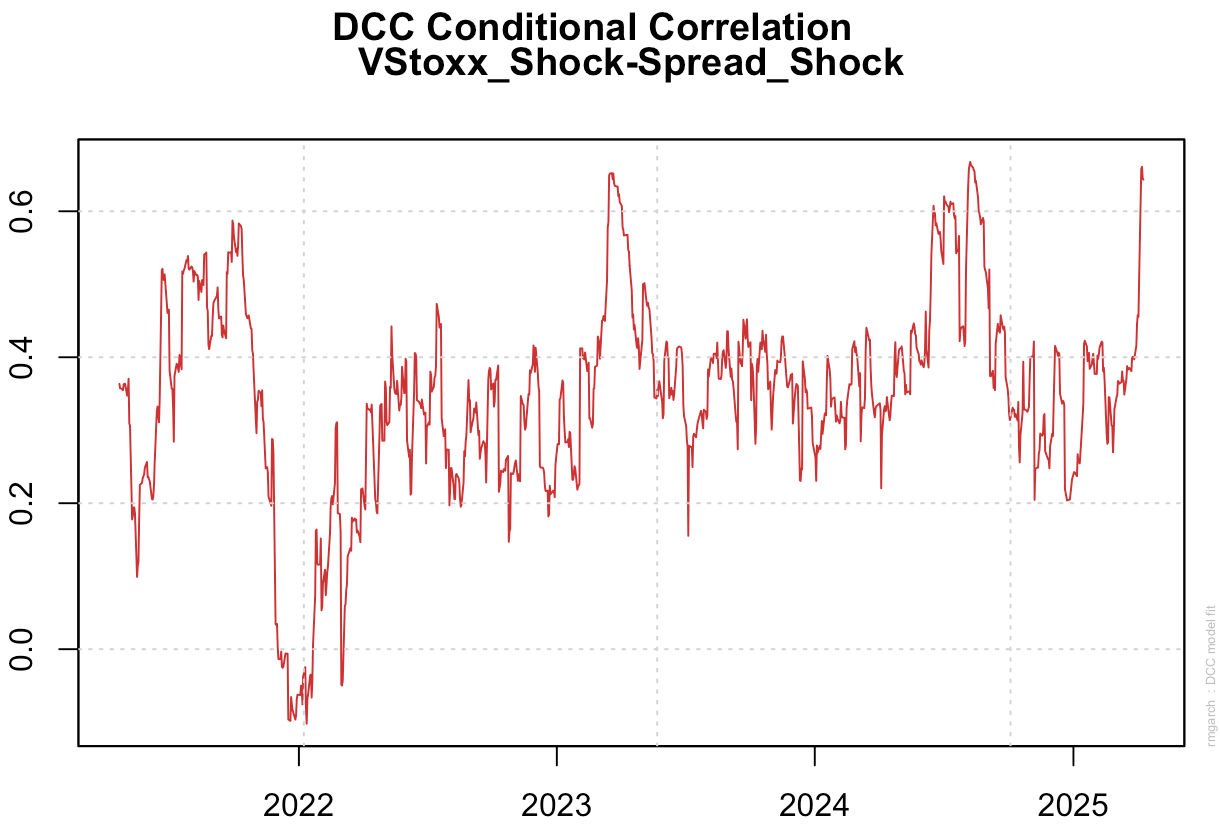




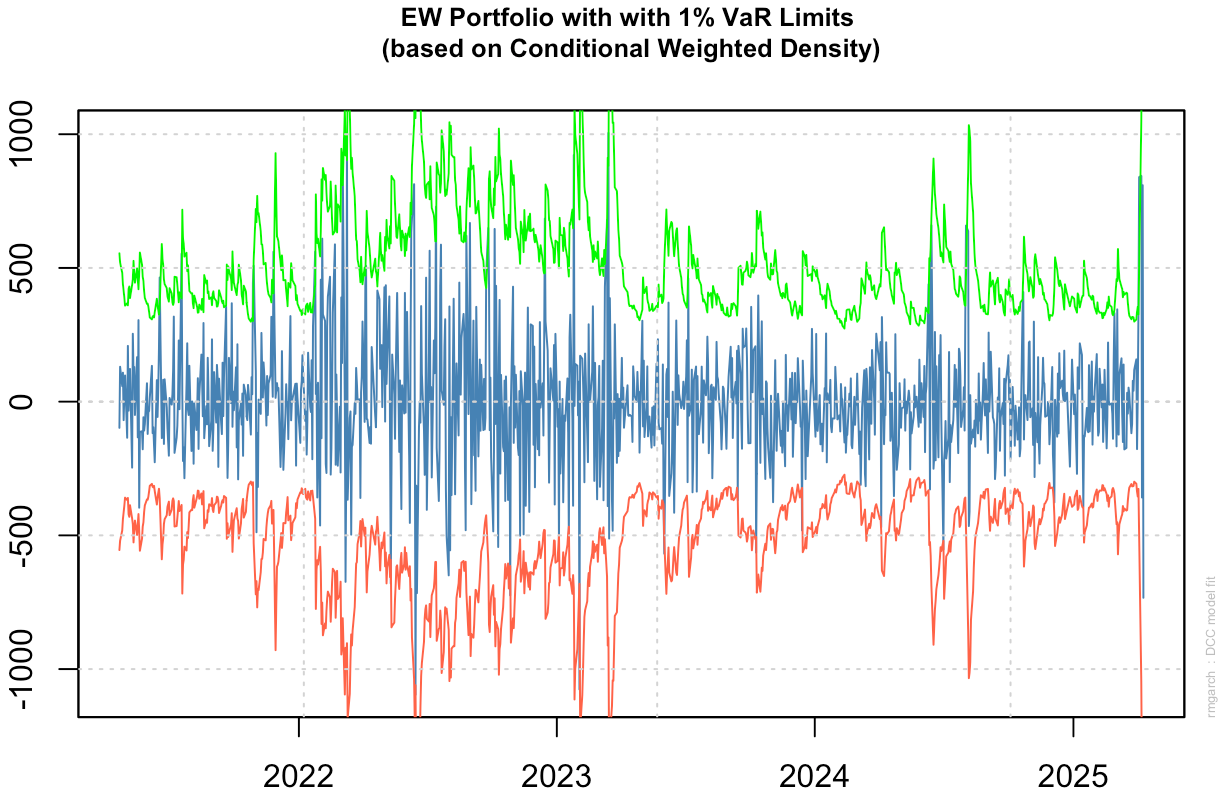

```
plot(fit_dcc, which=3)
```



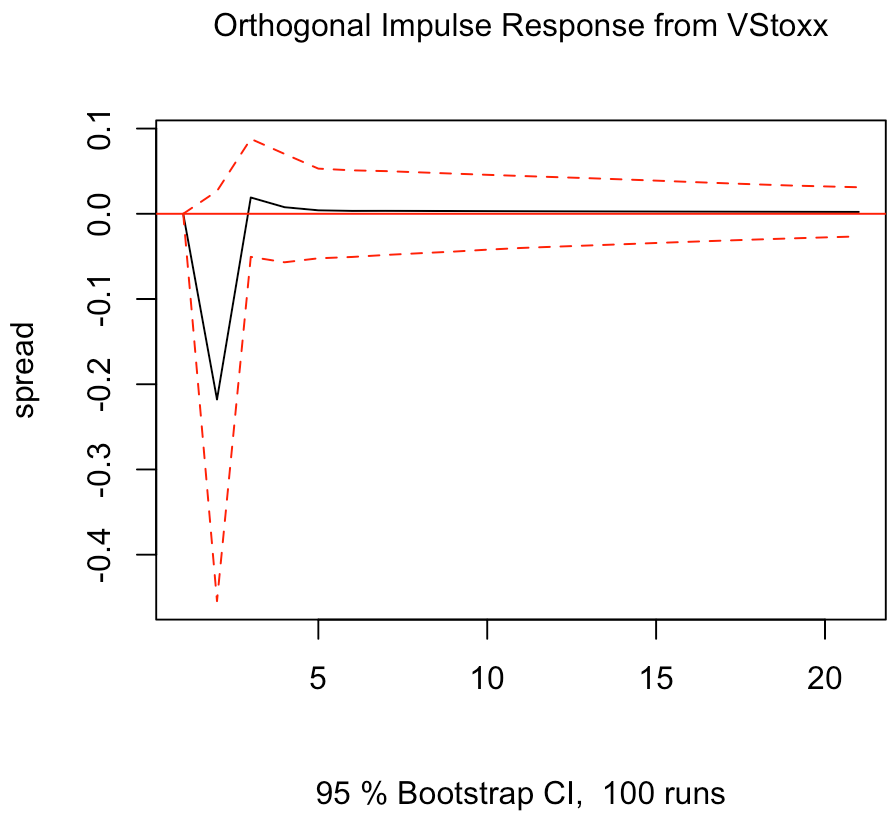

```
plot(fit_dcc, which=4)
```



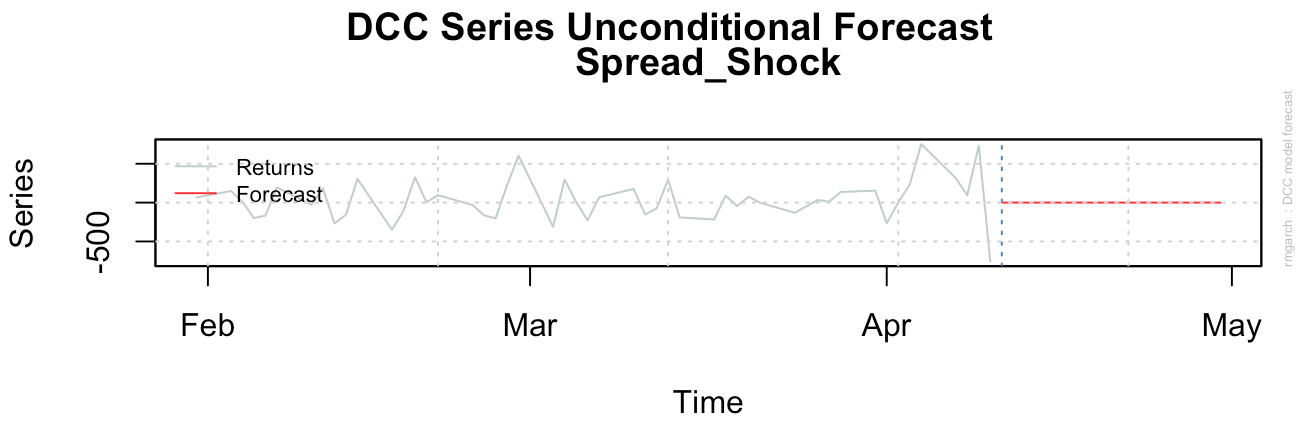

```
plot(fit_dcc, which=5)
```

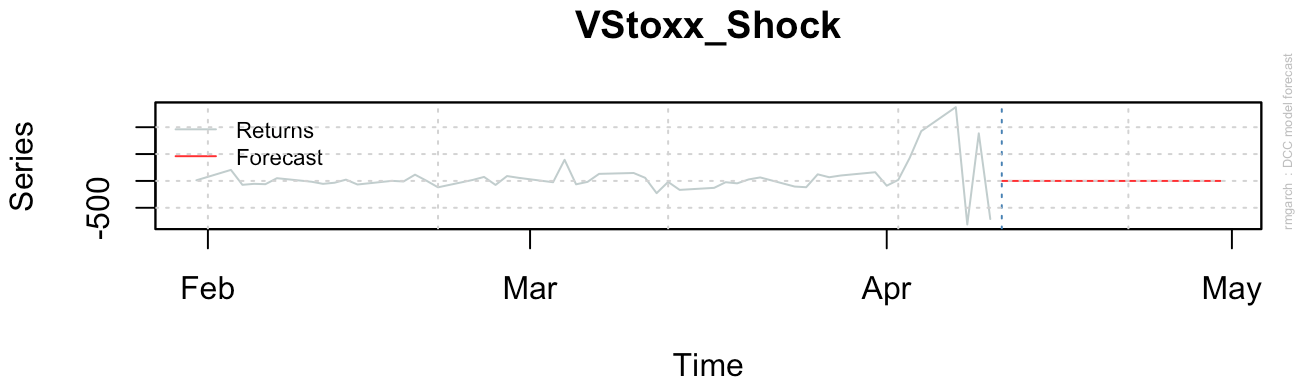



```
#colnames(var_fit$y)

# 2. VAR Impulse Response Functions (IRFs)
# This answers "how does the BTP-Bund spread respond to a VSTOXX shock?"
library(vars)
irf_result <- irf(var_fit, impulse = "VStoxx",
                  response = "spread",
                  n.ahead = 20, boot = TRUE)
plot(irf_result)
```

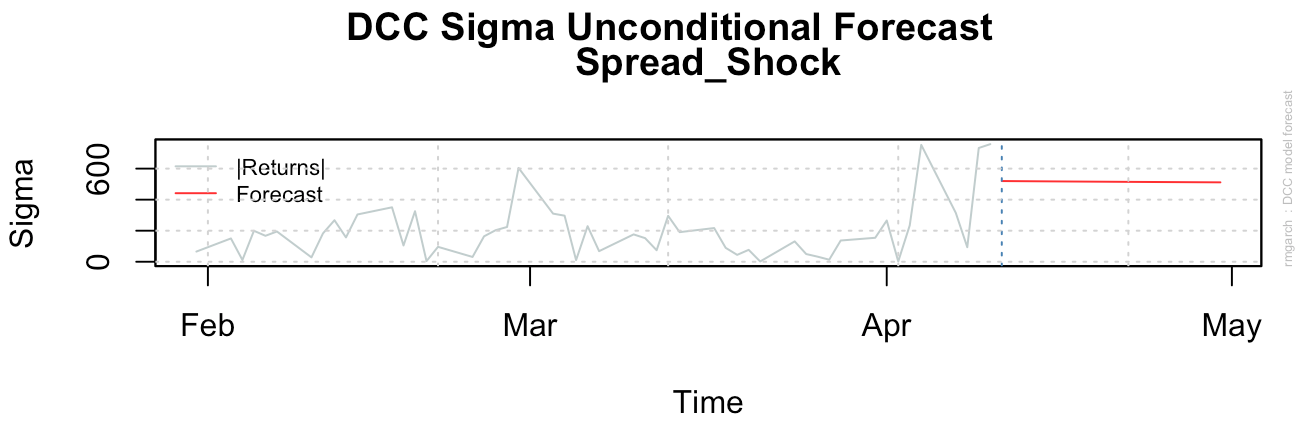


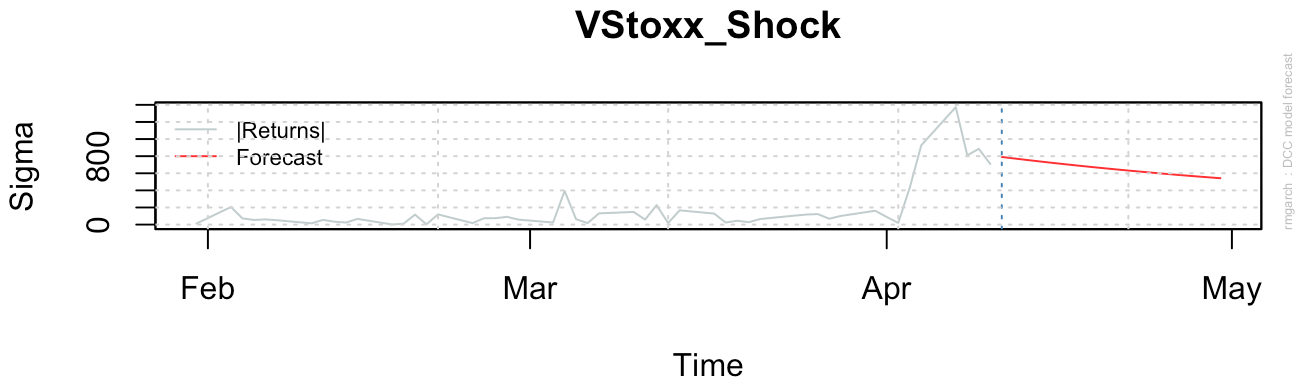

```
# 3. DCC-GARCH Volatility Forecast
# A short-horizon forecast of conditional variances and correlations
dcc_forecast <- dccforecast(fit_dcc, n.ahead = 20)
plot(dcc_forecast, which=1)
```



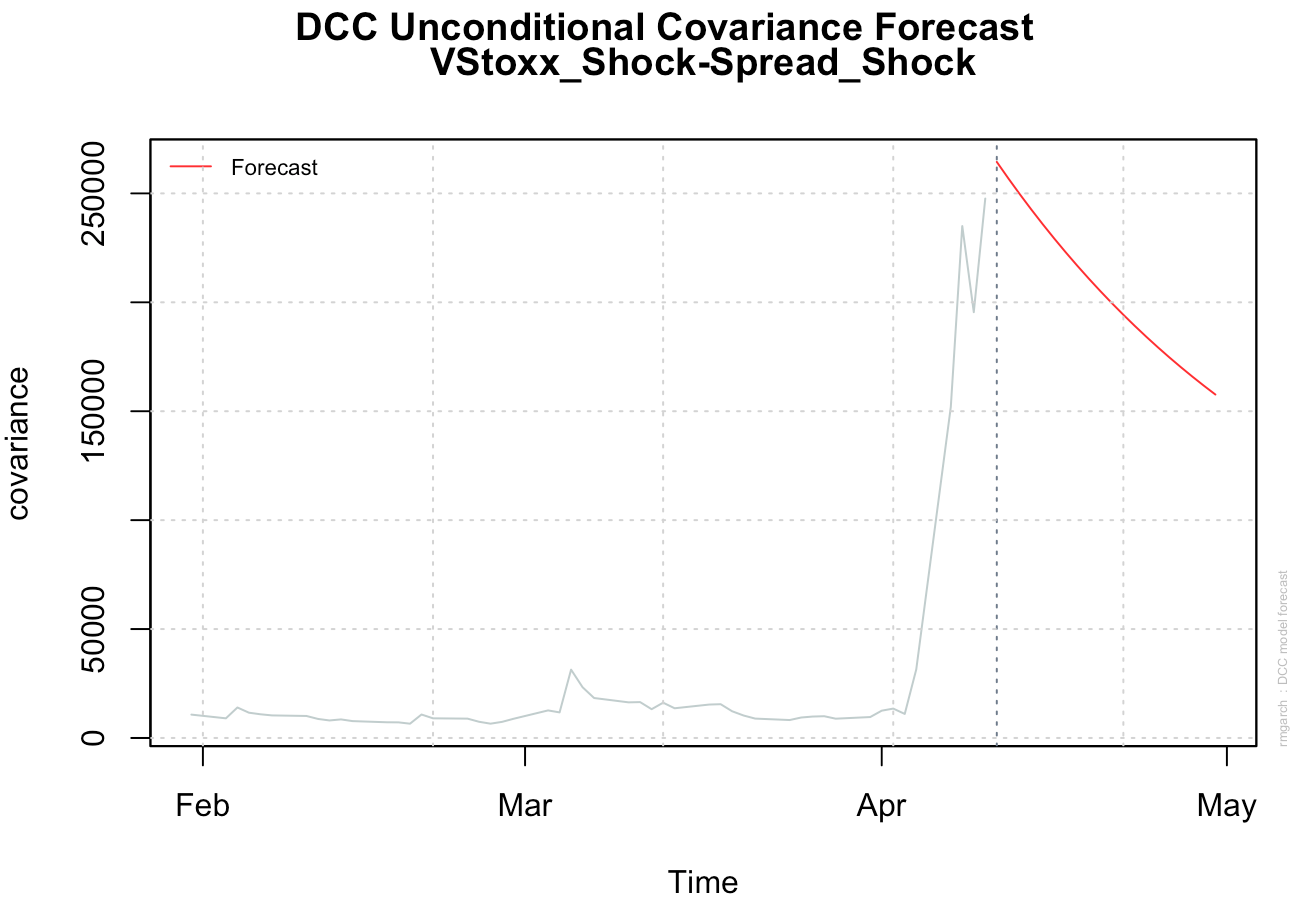



```
plot(dcc_forecast, which=2)
```






```
plot(dcc_forecast, which=3)
```



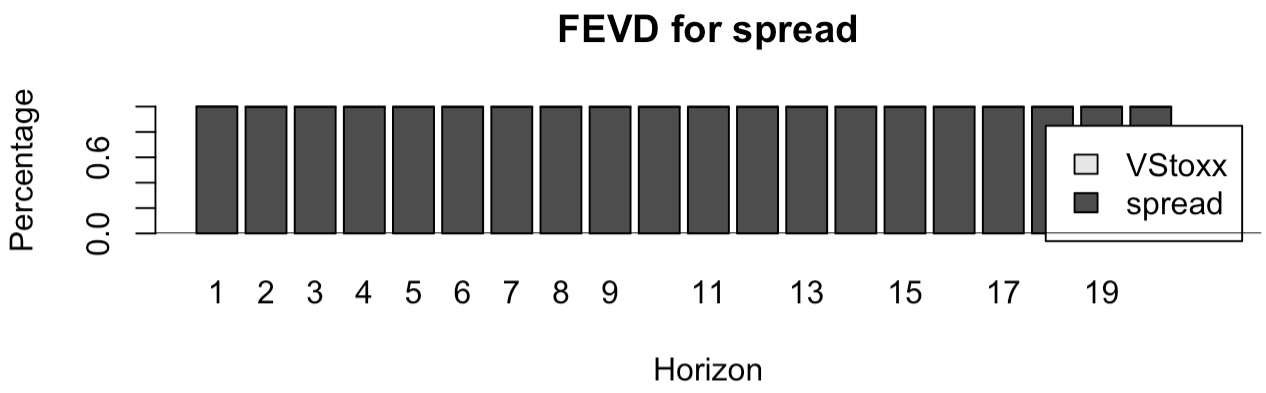

```
# Extract forecasted correlation
rcor(dcc_forecast)
```

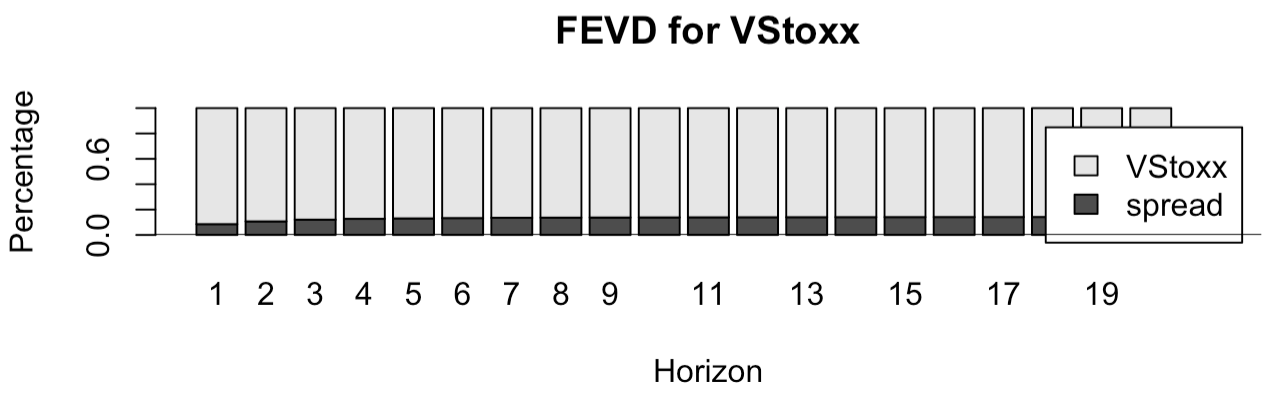
```
## $`2025-04-10`
## , , T+1
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6431675
## VStoxx_Shock    0.6431675    1.0000000
##
## , , T+2
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6386956
## VStoxx_Shock    0.6386956    1.0000000
##
## , , T+3
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6342889
## VStoxx_Shock    0.6342889    1.0000000
##
## , , T+4
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6299464
## VStoxx_Shock    0.6299464    1.0000000
##
## , , T+5
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6256671
## VStoxx_Shock    0.6256671    1.0000000
##
## , , T+6
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6214501
## VStoxx_Shock    0.6214501    1.0000000
##
## , , T+7
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6172944
## VStoxx_Shock    0.6172944    1.0000000
##
## , , T+8
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6131994
## VStoxx_Shock    0.6131994    1.0000000
##
## , , T+9
##
```

```
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6091639
## VStoxx_Shock    0.6091639    1.0000000
##
## , , T+10
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6051872
## VStoxx_Shock    0.6051872    1.0000000
##
## , , T+11
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.6012684
## VStoxx_Shock    0.6012684    1.0000000
##
## , , T+12
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.5974066
## VStoxx_Shock    0.5974066    1.0000000
##
## , , T+13
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.5936011
## VStoxx_Shock    0.5936011    1.0000000
##
## , , T+14
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock     1.000000     0.589851
## VStoxx_Shock     0.589851     1.000000
##
## , , T+15
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.5861556
## VStoxx_Shock    0.5861556    1.0000000
##
## , , T+16
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.5825139
## VStoxx_Shock    0.5825139    1.0000000
##
## , , T+17
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.5789252
## VStoxx_Shock    0.5789252    1.0000000
```

```
##
## , , T+18
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.5753888
## VStoxx_Shock    0.5753888    1.0000000
##
## , , T+19
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.5719039
## VStoxx_Shock    0.5719039    1.0000000
##
## , , T+20
##
##              Spread_Shock VStoxx_Shock
## Spread_Shock    1.0000000    0.5684697
## VStoxx_Shock    0.5684697    1.0000000
```

```
# 4. Variance Decomposition (FEVD)
# Tells you how much of the BTP-Bund spread variance is explained by VSTOXX shocks

fevd_result <- fevd(var_fit, n.ahead = 20)
plot(fevd_result)
```





Forecast

```
# Re-fit with out.sample to enable rolling forecast
dcc_fit2 <- dccfit(dcc_spec, data = resids,
                   out.sample = 100)   # hold out last 100 obs

# 1. Unconditional n-ahead forecast
dcc_fore <- dccforecast(dcc_fit2, n.ahead = 20)

# Forecasted correlation
rcor(dcc_fore)   # 2x2xT array
```

```
## $`1972-07-05`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2921265
## VStoxx 0.2921265 1.0000000
##
## , , T+2
##
##          spread   VStoxx
## spread 1.000000 0.292504
## VStoxx 0.292504 1.000000
##
## , , T+3
##
##           spread    VStoxx
## spread 1.0000000 0.2928766
## VStoxx 0.2928766 1.0000000
##
## , , T+4
##
##           spread    VStoxx
## spread 1.0000000 0.2932443
## VStoxx 0.2932443 1.0000000
##
## , , T+5
##
##           spread    VStoxx
## spread 1.0000000 0.2936073
## VStoxx 0.2936073 1.0000000
##
## , , T+6
##
##           spread    VStoxx
## spread 1.0000000 0.2939655
## VStoxx 0.2939655 1.0000000
##
## , , T+7
##
##           spread    VStoxx
## spread 1.0000000 0.2943191
## VStoxx 0.2943191 1.0000000
##
## , , T+8
##
##          spread   VStoxx
## spread 1.000000 0.294668
## VStoxx 0.294668 1.000000
##
## , , T+9
##
```

```
##           spread    VStoxx
## spread 1.0000000 0.2950125
## VStoxx 0.2950125 1.0000000
##
## , , T+10
##
##           spread    VStoxx
## spread 1.0000000 0.2953524
## VStoxx 0.2953524 1.0000000
##
## , , T+11
##
##           spread    VStoxx
## spread 1.0000000 0.2956879
## VStoxx 0.2956879 1.0000000
##
## , , T+12
##
##          spread   VStoxx
## spread 1.000000 0.296019
## VStoxx 0.296019 1.000000
##
## , , T+13
##
##           spread    VStoxx
## spread 1.0000000 0.2963459
## VStoxx 0.2963459 1.0000000
##
## , , T+14
##
##           spread    VStoxx
## spread 1.0000000 0.2966684
## VStoxx 0.2966684 1.0000000
##
## , , T+15
##
##           spread    VStoxx
## spread 1.0000000 0.2969868
## VStoxx 0.2969868 1.0000000
##
## , , T+16
##
##          spread   VStoxx
## spread 1.000000 0.297301
## VStoxx 0.297301 1.000000
##
## , , T+17
##
##           spread    VStoxx
## spread 1.0000000 0.2976112
## VStoxx 0.2976112 1.0000000
```

```
##
## , , T+18
##
##           spread    VStoxx
## spread 1.0000000 0.2979172
## VStoxx 0.2979172 1.0000000
##
## , , T+19
##
##           spread    VStoxx
## spread 1.0000000 0.2982194
## VStoxx 0.2982194 1.0000000
##
## , , T+20
##
##           spread    VStoxx
## spread 1.0000000 0.2985175
## VStoxx 0.2985175 1.0000000
```

```
# Forecasted volatilities
sigma(dcc_fore)
```

```
## , , 1972-07-05
##
##        spread   VStoxx
## T+1  3.484710 1.077099
## T+2  3.520049 1.166578
## T+3  3.554256 1.248303
## T+4  3.587383 1.323734
## T+5  3.619475 1.393913
## T+6  3.650575 1.459616
## T+7  3.680725 1.521442
## T+8  3.709963 1.579863
## T+9  3.738326 1.635260
## T+10 3.765847 1.687947
## T+11 3.792559 1.738184
## T+12 3.818492 1.786193
## T+13 3.843675 1.832162
## T+14 3.868136 1.876253
## T+15 3.891901 1.918610
## T+16 3.914994 1.959356
## T+17 3.937439 1.998602
## T+18 3.959259 2.036445
## T+19 3.980474 2.072973
## T+20 4.001106 2.108265
```

```
# 2. Rolling 1-step-ahead forecast (more meaningful for evaluation)
dcc_roll <- dccforecast(dcc_fit2, n.ahead = 1, n.roll = 99)
rcor(dcc_roll)
```

```
## $`1972-07-05`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2921265
## VStoxx 0.2921265 1.0000000
##
##
## $`1972-07-06`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2889044
## VStoxx 0.2889044 1.0000000
##
##
## $`1972-07-07`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2915303
## VStoxx 0.2915303 1.0000000
##
##
## $`1972-07-08`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.316604
## VStoxx 0.316604 1.000000
##
##
## $`1972-07-09`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3191784
## VStoxx 0.3191784 1.0000000
##
##
## $`1972-07-10`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2667474
## VStoxx 0.2667474 1.0000000
##
##
## $`1972-07-11`
## , , T+1
##
```

```
##          spread   VStoxx
## spread 1.000000 0.257887
## VStoxx 0.257887 1.000000
##
##
## $`1972-07-12`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2551105
## VStoxx 0.2551105 1.0000000
##
##
## $`1972-07-13`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2539732
## VStoxx 0.2539732 1.0000000
##
##
## $`1972-07-14`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2415038
## VStoxx 0.2415038 1.0000000
##
##
## $`1972-07-15`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.272398
## VStoxx 0.272398 1.000000
##
##
## $`1972-07-16`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2905147
## VStoxx 0.2905147 1.0000000
##
##
## $`1972-07-17`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2870287
## VStoxx 0.2870287 1.0000000
```

```
##
##
## $`1972-07-18`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3334976
## VStoxx 0.3334976 1.0000000
##
##
## $`1972-07-19`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.374725
## VStoxx 0.374725 1.000000
##
##
## $`1972-07-20`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4184694
## VStoxx 0.4184694 1.0000000
##
##
## $`1972-07-21`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4090321
## VStoxx 0.4090321 1.0000000
##
##
## $`1972-07-22`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4042813
## VStoxx 0.4042813 1.0000000
##
##
## $`1972-07-23`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4089282
## VStoxx 0.4089282 1.0000000
##
##
## $`1972-07-24`
```

```
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4058091
## VStoxx 0.4058091 1.0000000
##
##
## $`1972-07-25`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3493189
## VStoxx 0.3493189 1.0000000
##
##
## $`1972-07-26`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3345448
## VStoxx 0.3345448 1.0000000
##
##
## $`1972-07-27`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.338574
## VStoxx 0.338574 1.000000
##
##
## $`1972-07-28`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3369999
## VStoxx 0.3369999 1.0000000
##
##
## $`1972-07-29`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3325391
## VStoxx 0.3325391 1.0000000
##
##
## $`1972-07-30`
## , , T+1
##
##           spread    VStoxx
```

```
## spread 1.0000000 0.2093612
## VStoxx 0.2093612 1.0000000
##
##
## $`1972-07-31`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.1942542
## VStoxx 0.1942542 1.0000000
##
##
## $`1972-08-01`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.195158
## VStoxx 0.195158 1.000000
##
##
## $`1972-08-02`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.221588
## VStoxx 0.221588 1.000000
##
##
## $`1972-08-03`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.231355
## VStoxx 0.231355 1.000000
##
##
## $`1972-08-04`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2302295
## VStoxx 0.2302295 1.0000000
##
##
## $`1972-08-05`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2252778
## VStoxx 0.2252778 1.0000000
##
```

```
##
## $`1972-08-06`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2555651
## VStoxx 0.2555651 1.0000000
##
##
## $`1972-08-07`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2440065
## VStoxx 0.2440065 1.0000000
##
##
## $`1972-08-08`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2475448
## VStoxx 0.2475448 1.0000000
##
##
## $`1972-08-09`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2423314
## VStoxx 0.2423314 1.0000000
##
##
## $`1972-08-10`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2920128
## VStoxx 0.2920128 1.0000000
##
##
## $`1972-08-11`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3096118
## VStoxx 0.3096118 1.0000000
##
##
## $`1972-08-12`
## , , T+1
```

```
##
##           spread    VStoxx
## spread 1.0000000 0.3283234
## VStoxx 0.3283234 1.0000000
##
##
## $`1972-08-13`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4129468
## VStoxx 0.4129468 1.0000000
##
##
## $`1972-08-14`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4173719
## VStoxx 0.4173719 1.0000000
##
##
## $`1972-08-15`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4097182
## VStoxx 0.4097182 1.0000000
##
##
## $`1972-08-16`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3892511
## VStoxx 0.3892511 1.0000000
##
##
## $`1972-08-17`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3931767
## VStoxx 0.3931767 1.0000000
##
##
## $`1972-08-18`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4010427
```

```
## VStoxx 0.4010427 1.0000000
##
##
## $`1972-08-19`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3789626
## VStoxx 0.3789626 1.0000000
##
##
## $`1972-08-20`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.385068
## VStoxx 0.385068 1.000000
##
##
## $`1972-08-21`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3938369
## VStoxx 0.3938369 1.0000000
##
##
## $`1972-08-22`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4004272
## VStoxx 0.4004272 1.0000000
##
##
## $`1972-08-23`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3873616
## VStoxx 0.3873616 1.0000000
##
##
## $`1972-08-24`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.372558
## VStoxx 0.372558 1.000000
##
##
```

```
## $`1972-08-25`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3723217
## VStoxx 0.3723217 1.0000000
##
##
## $`1972-08-26`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3982304
## VStoxx 0.3982304 1.0000000
##
##
## $`1972-08-27`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3938967
## VStoxx 0.3938967 1.0000000
##
##
## $`1972-08-28`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4002289
## VStoxx 0.4002289 1.0000000
##
##
## $`1972-08-29`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4095006
## VStoxx 0.4095006 1.0000000
##
##
## $`1972-08-30`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.417816
## VStoxx 0.417816 1.000000
##
##
## $`1972-08-31`
## , , T+1
##
```

```
##          spread   VStoxx
## spread 1.000000 0.416781
## VStoxx 0.416781 1.000000
##
##
## $`1972-09-01`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3759753
## VStoxx 0.3759753 1.0000000
##
##
## $`1972-09-02`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3817074
## VStoxx 0.3817074 1.0000000
##
##
## $`1972-09-03`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3653586
## VStoxx 0.3653586 1.0000000
##
##
## $`1972-09-04`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2861675
## VStoxx 0.2861675 1.0000000
##
##
## $`1972-09-05`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2715427
## VStoxx 0.2715427 1.0000000
##
##
## $`1972-09-06`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2738051
## VStoxx 0.2738051 1.0000000
```

```
##
##
## $`1972-09-07`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.341007
## VStoxx 0.341007 1.000000
##
##
## $`1972-09-08`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3406997
## VStoxx 0.3406997 1.0000000
##
##
## $`1972-09-09`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2996274
## VStoxx 0.2996274 1.0000000
##
##
## $`1972-09-10`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2986106
## VStoxx 0.2986106 1.0000000
##
##
## $`1972-09-11`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2609831
## VStoxx 0.2609831 1.0000000
##
##
## $`1972-09-12`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.2888583
## VStoxx 0.2888583 1.0000000
##
##
## $`1972-09-13`
```

```
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3230025
## VStoxx 0.3230025 1.0000000
##
##
## $`1972-09-14`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3409965
## VStoxx 0.3409965 1.0000000
##
##
## $`1972-09-15`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3448649
## VStoxx 0.3448649 1.0000000
##
##
## $`1972-09-16`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3662423
## VStoxx 0.3662423 1.0000000
##
##
## $`1972-09-17`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3650046
## VStoxx 0.3650046 1.0000000
##
##
## $`1972-09-18`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3621398
## VStoxx 0.3621398 1.0000000
##
##
## $`1972-09-19`
## , , T+1
##
##          spread   VStoxx
```

```
## spread 1.000000 0.364542
## VStoxx 0.364542 1.000000
##
##
## $`1972-09-20`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.378826
## VStoxx 0.378826 1.000000
##
##
## $`1972-09-21`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3655359
## VStoxx 0.3655359 1.0000000
##
##
## $`1972-09-22`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3667989
## VStoxx 0.3667989 1.0000000
##
##
## $`1972-09-23`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3472514
## VStoxx 0.3472514 1.0000000
##
##
## $`1972-09-24`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3689952
## VStoxx 0.3689952 1.0000000
##
##
## $`1972-09-25`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.386377
## VStoxx 0.386377 1.000000
##
```

```
##
## $`1972-09-26`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3816395
## VStoxx 0.3816395 1.0000000
##
##
## $`1972-09-27`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3824117
## VStoxx 0.3824117 1.0000000
##
##
## $`1972-09-28`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3836755
## VStoxx 0.3836755 1.0000000
##
##
## $`1972-09-29`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3792341
## VStoxx 0.3792341 1.0000000
##
##
## $`1972-09-30`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3976847
## VStoxx 0.3976847 1.0000000
##
##
## $`1972-10-01`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.3991887
## VStoxx 0.3991887 1.0000000
##
##
## $`1972-10-02`
## , , T+1
```

```
##
##           spread    VStoxx
## spread 1.0000000 0.3974015
## VStoxx 0.3974015 1.0000000
##
##
## $`1972-10-03`
## , , T+1
##
##          spread   VStoxx
## spread 1.000000 0.395514
## VStoxx 0.395514 1.000000
##
##
## $`1972-10-04`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4143981
## VStoxx 0.4143981 1.0000000
##
##
## $`1972-10-05`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4453301
## VStoxx 0.4453301 1.0000000
##
##
## $`1972-10-06`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4574396
## VStoxx 0.4574396 1.0000000
##
##
## $`1972-10-07`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.4550373
## VStoxx 0.4550373 1.0000000
##
##
## $`1972-10-08`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.5117068
```

```
## VStoxx 0.5117068 1.0000000
##
##
## $`1972-10-09`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.6598626
## VStoxx 0.6598626 1.0000000
##
##
## $`1972-10-10`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.6647932
## VStoxx 0.6647932 1.0000000
##
##
## $`1972-10-11`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.6491839
## VStoxx 0.6491839 1.0000000
##
##
## $`1972-10-12`
## , , T+1
##
##           spread    VStoxx
## spread 1.0000000 0.6451056
## VStoxx 0.6451056 1.0000000
```

```
# 1-Step Ahead Forecast with VAR
library(vars)

# Forecast 1 step ahead (next trading day)
var_forecast <- predict(var_fit, n.ahead = 1, ci = 0.95)

# Print the forecast
print(var_forecast)
```

```
## $spread
##                  fcst     lower    upper       CI
## spread.fcst 0.5319803 -7.109641 8.173602 7.641622
##
## $VStoxx
##                 fcst    lower    upper       CI
## VStoxx.fcst 37.29659 33.97837 40.61481 3.318222
```

```
last_spread <- tail(spread, 1)
forecast_level <- last_spread + var_forecast$fcst$spread[1, "fcst"]
forecast_level
```

```
##              spread
## 2026-04-17 71.53198
```

Test the model

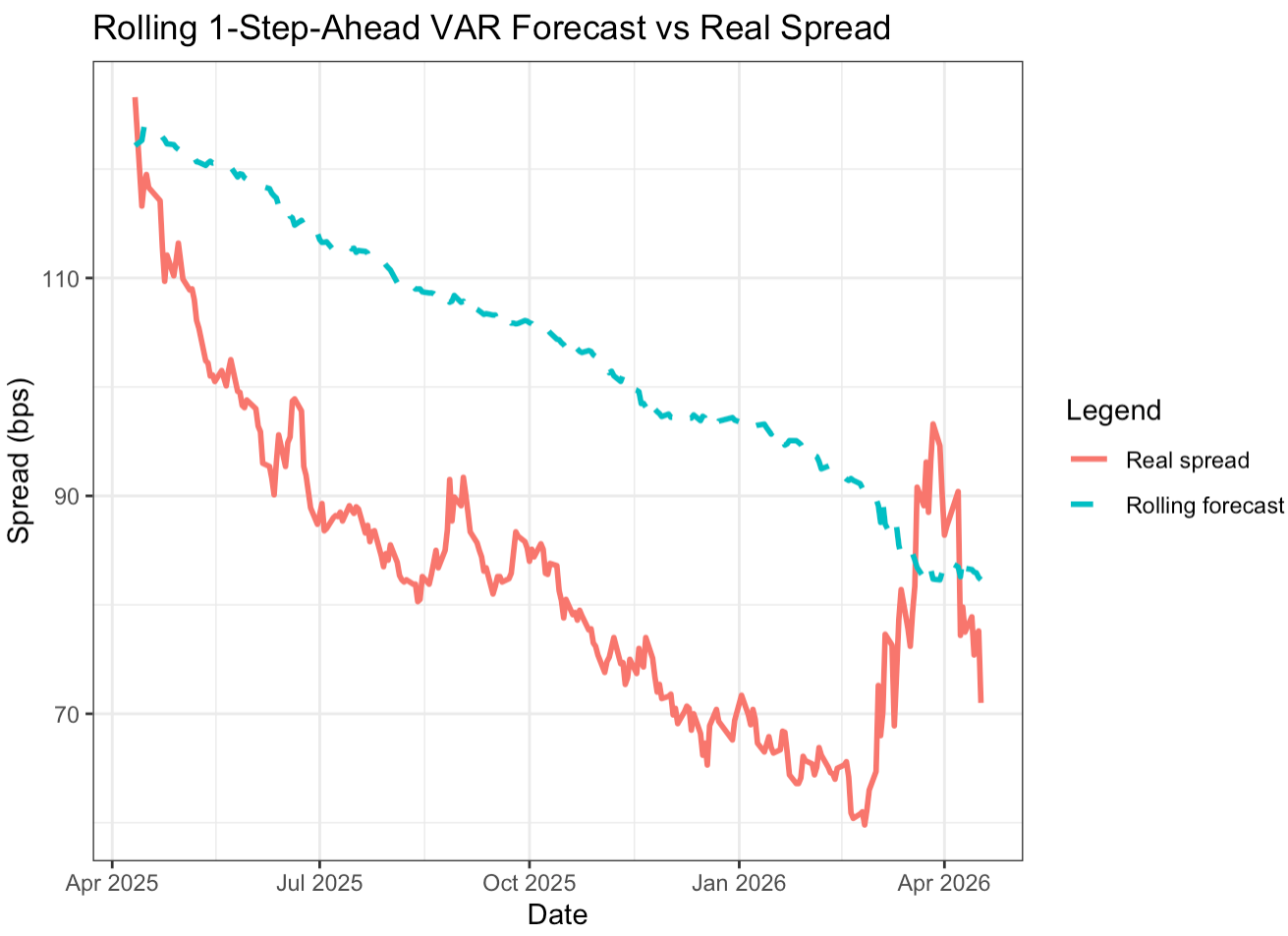
We now use the test portion of our data, to see the accuracy of our training model

```
library(vars)
library(xts)
library(ggplot2)

# ==========================
# Test set: rolling 1-step-ahead out-of-sample forecast
# ==========================

train_multi <- train
test_multi  <- test

# 1. Selezione lag ottimale sul train
lag_sel <- VARselect(train_multi, lag.max = 10, type = "const")
p_opt <- as.numeric(lag_sel$selection["AIC(n)"])

cat("Optimal lag selected on train:", p_opt, "\n")
```

```
## Optimal lag selected on train: 2
```

```
# 2. Rolling 1-step-ahead forecast sulla serie differenziata "spread"
n_test <- nrow(test_multi)

fc_dspread <- numeric(n_test)
real_dspread <- as.numeric(test_multi[, "spread"])

# Among the candidate rolling window sizes, 200 observations produced the lowest out-
of-sample forecast error and  # the most plausible final spread estimate, therefore i
t was selected as the preferred specification.
window_size <- 250

for (i in 1:n_test) {

  if (i == 1) {
    available_data <- train_multi
  } else {
    available_data <- rbind(train_multi, test_multi[1:(i - 1), ])
  }

  if (nrow(available_data) > window_size) {
    current_train <- available_data[(nrow(available_data) - window_size + 1):nrow(ava
ilable_data), ]
  } else {
    current_train <- available_data
  }

  var_roll <- VAR(current_train, p = p_opt, type = "const")
  pred_1 <- predict(var_roll, n.ahead = 1, ci = 0.95)

  fc_dspread[i] <- as.numeric(pred_1$fcst$spread[1, "fcst"])
}


# 3. Date del test
test_dates <- index(test_multi)

# 4. Ricostruzione dello spread in livelli
# ultimo valore osservato in livelli prima dell'inizio del test
test_start_date <- test_dates[1]
pos_test_start <- which(index(spread) == test_start_date)
last_level_before_test <- as.numeric(spread[pos_test_start - 1])

# forecast in livelli
fc_level <- last_level_before_test + cumsum(fc_dspread)

# valori reali in livelli
real_level <- as.numeric(spread[test_dates])

# 5. Data frame finale
df_roll <- data.frame(
  Date = as.Date(test_dates),
```

```
  RealSpread = real_level,
  ForecastSpread = fc_level,
  RealDiff = real_dspread,
  ForecastDiff = fc_dspread
)

# 6. Grafico livelli
ggplot(df_roll, aes(x = Date)) +
  geom_line(aes(y = RealSpread, color = "Real spread"), linewidth = 1) +
  geom_line(aes(y = ForecastSpread, color = "Rolling forecast"),
            linewidth = 1, linetype = "dashed") +
  labs(
    title = "Rolling 1-Step-Ahead VAR Forecast vs Real Spread",
    y = "Spread (bps)",
    x = "Date",
    color = "Legend"
  ) +
  theme_bw()
```



```
# 7. Metriche sui livelli
err_level <- df_roll$RealSpread - df_roll$ForecastSpread

mae_level <- mean(abs(err_level), na.rm = TRUE)
rmse_level <- sqrt(mean(err_level^2, na.rm = TRUE))

cat("MAE levels:", round(mae_level, 4), "\n")
```

```
## MAE levels: 21.6974
```

```
cat("RMSE levels:", round(rmse_level, 4), "\n")
```

```
## RMSE levels: 22.8775
```

```
# 8. Benchmark naive
naive_fc_level <- rep(last_level_before_test, length(fc_level))

mae_naive <- mean(abs(real_level - naive_fc_level), na.rm = TRUE)
rmse_naive <- sqrt(mean((real_level - naive_fc_level)^2, na.rm = TRUE))

cat("Naive MAE:", round(mae_naive, 4), "\n")
```

```
## Naive MAE: 39.3984
```

```
cat("Naive RMSE:", round(rmse_naive, 4), "\n")
```

```
## Naive RMSE: 41.5837
```

```
# 9. Valore finale out-of-sample 1-step-ahead
final_forecast_date <- tail(test_dates, 1)
final_forecast_diff <- tail(fc_dspread, 1)
final_forecast_level <- tail(fc_level, 1)

cat("Final forecast date:", as.character(final_forecast_date), "\n")
```

```
## Final forecast date: 2026-04-17
```

```
cat("Final out-of-sample 1-step forecast (diff):", round(final_forecast_diff, 4),
"\n")
```

```
## Final out-of-sample 1-step forecast (diff): -0.202
```

```
cat("Final out-of-sample 1-step forecast (level):", round(final_forecast_level, 4),
"\n")
```

```
## Final out-of-sample 1-step forecast (level): 82.303
```